

Approximation Algorithms

STEP 2: What is an Approximation Algorithm?

It's just an algorithm that:

- Runs in polynomial time
- Doesn't give the exact optimal solution
- BUT gives a solution that is provably close to optimal

The word **provably** is super important – we are not guessing; we mathematically guarantee that it won't be too bad.

STEP 3: Approximation Ratio (the HEART of this topic)

Take a breath – this part is important but we'll go gently.

Let's denote:

- **ALG** = solution given by our approximation algorithm
- **OPT** = value of the optimal solution (the best possible)

Depending on the type of problem:

✓ Minimization (example: Vertex Cover, Set Cover)

Smaller is better

The algorithm should satisfy:

$$ALG \leq \rho(n) \cdot OPT$$

If $\rho(n) = 2$, that means:

"The algorithm's answer is at most 2 times the optimal."

✓ Maximization (example: Max-SAT)

Bigger is better

Here:

$$ALG \geq \frac{1}{\rho(n)} \cdot OPT$$

If $\rho(n) = 2$, that means:

"The algorithm achieves at least half of the optimal."

The approximation ratio tells you:

How bad the algorithm can be in the worst case.

Approximation Algorithm for Vertex Cover

- Select an arbitrary edge e
- Add the vertices u and v of the edge e to the result

- Remove e and all the edges from u and v from the graph
- Repeat until the graph is empty

$$|ALG| \leq 2|OPT|$$

Approximation Algorithm for Set Cover

- Take the set with most number of uncovered elements
- Mark the elements in this set as covered
- Repeat until all elements are covered

$$|ALG| \leq H_n |OPT|$$

$$H_n \approx \ln(n)$$

Proof

- Let $OPT = k$, i.e., the optimal solution uses **k sets** to cover all n elements
- On average, OPT covers n/k elements per set. (This doesn't mean every OPT set does this – but on average yes.)
- At ANY moment while running the greedy algorithm, there are still some uncovered elements left.
- Let's say at some step there are **t uncovered elements**.
- Since OPT used k sets to cover everything,
- OPT could cover these t remaining elements with **at most k sets**.

- So one of those sets must cover at least t/k elements.
- Because:
If k sets cover t elements, average coverage per set $= \frac{t}{k}$
- There must be one set with $\geq$ average coverage.
- OPT has a set covering $\geq t/k$ uncovered elements
- Greedy picks the set covering the **maximum** uncovered elements
👉 So greedy will also cover **at least t/k elements.**
- Now look at how t decreases
Start with:
- $t = n$
After picking one set, we cover $\geq n/k$ elements.
Remaining uncovered:

$$n_1 \leq n - \frac{n}{k}$$

$$n_1 \leq n(1 - \frac{1}{k})$$

$$n_2 \leq n_1(1 - \frac{1}{k})$$

$$n_2 \leq n(1 - \frac{1}{k})^2$$

$$n_i \leq n(1 - \frac{1}{k})^i$$

we need to find for what i n_i becomes < 1

$$(1 - \frac{1}{k})^k \approx \frac{1}{e}$$

$$n(1 - \frac{1}{k})^i = 1$$

using these,

we need $k \ln(n)$ steps

$$|ALG| \leq k \ln(n)$$

$$|ALG| \leq \ln(n) |OPT|$$